

Documentazione

”Rompicapo vitaminico”

Versione: 1.0

Data: 8 settembre 2025

Autori: Federico rigolon, Luca Milioli

Email: 318199@studenti.unimore.it, 317076@studenti.unimore.it

Indice

1	Introduzione	2
1.1	Scopo del documento	2
1.2	Obiettivi del progetto	2
1.3	Pubblico di riferimento	2
2	Differenza tra le Versioni	3
2.1	Versione 1.0	3
2.2	Versione 2.0	3
2.3	Tabella comparativa	3
3	Separazione della Logica (GUI/Business Logic)	4
3.1	Principi architetturali	4
3.2	Livello di presentazione (GUI)	4
3.2.1	Componenti dell'interfaccia	4
3.2.2	Gestione dei segnali	4
3.3	Livello di business logic	4
3.3.1	Moduli principali	4
3.3.2	Pattern utilizzati	5
3.3.3	Modelli	5
3.4	Interfaccia di comunicazione	5
4	Lettura dei Dati (CSV)	6
4.1	Formato dei file supportati	6
4.2	Processo di lettura	6
5	Panoramica Generale	7
5.1	Architettura del sistema	7
5.2	Dipendenze esterne	7
6	Esportazione	8
6.1	Web build personalizzata	8
6.2	Funzionalità della shell	8
6.3	Processo di esportazione	8
6.4	Formati di output	9
7	Rilascio	10
7.1	Git repo	10

1 Introduzione

Panoramica: Questa sezione fornisce un'introduzione generale al progetto.

1.1 Scopo del documento

Lo scopo del documento è quello di dare dettagli implementativi sullo sviluppo del gioco.

1.2 Obiettivi del progetto

Sviluppo di un "risolvi l'equazione" a tema frutta e verdura.

1.3 Pubblico di riferimento

Il gioco è pensato per un'utenza infantile, è infatti pensato come gioco educativo a tema frutta e verdura per studenti delle elementari.

2 Differenza tra le Versioni

Nota: In questa sezione vengono analizzate le principali differenze tra le diverse versioni del software.

2.1 Versione 1.0

La versione 1.0 (branch 1.0 su github) è pensata per un utilizzo desktop, contiene infatti un menu iniziale, non è pensata per l'esportazione web: mancano infatti le funzioni legate alla comunicazione con il sito (http requests), inoltre alcune modifiche richieste dal cliente non sono state implementate in questa versione.

2.2 Versione 2.0

La versione 2.0 è quella finale, è pensata per l'export sul sito web. Aggiunge funzioni JavaScript, pulsante fullscreen, rimuove il menu iniziale e ha una serie di migliorie e bugfix non presenti nella prima versione.

2.3 Tabella comparativa

Funzionalità	v1.0	v2.0
Menu iniziale	✓	✗
Funzionalità Desktop	✓	✗
Funzionalità web	✗	✓
Migliorie varie	✗	✓

Tabella 1: Confronto delle funzionalità tra versioni

3 Separazione della Logica (GUI/Business Logic)

Architettura: Questa sezione descrive come è stata implementata la separazione tra interfaccia utente e logica di business.

3.1 Principi architetturali

Abbiamo tenuto separate l'interfaccia utente dalla logica tramite una serie di classi che rappresentano vari oggetti presenti nel gioco.

3.2 Livello di presentazione (GUI)

3.2.1 Componenti dell'interfaccia

- EquationContainer, è un HBoxContainer che dispone in riga gli oggetti: Fruit, Rhs, Sign.
- Fruit, è un semplice TextureRect che rappresenta un frutto.
- Rhs, è una semplice label che rappresenta il risultato dell'equazione.
- Sign, è un semplice TextureRect che rappresenta un segno (+, -, *, /).
- Blackboard, è un VBoxContainer che rappresenta la lavagna, viene istanziato un EquationContainer per ogni equazione presente nel sistema di equazioni.
- Keyboard, è la scena che rappresenta la tastiera virtuale che appare quando l'utente vuole inserire la risposta.
- Gui, è la scena gui principale, mette insieme tutti gli oggetti sopra descritti e aggiunge topbar, background e alcuni bottoni.

3.2.2 Gestione dei segnali

Quando l'utente preme il bottone di inserimento, viene emesso un segnale che permette di far apparire la tastiera. Allo stesso modo, quando l'utente inserisce il valore, viene intercettato dalla Keyboard il segnale opportuno. La keyboard aggiorna poi il testo della label. Esistono ulteriori segnali come `child_appeared`, `animation_done` e `kill_me` che servono per far uscire di scena i componenti e gestire al meglio le animazioni.

3.3 Livello di business logic

La logica è contenuta dentro la GameLogic.

3.3.1 Moduli principali

GameLogic, SystemEquationFactory.

3.3.2 Pattern utilizzati

sia GameLogic che SystemEquationFactory sono singleton autoload globali.

3.3.3 Modelli

- Equation, è una classe che rappresenta un'equazione utilizzando un array di coefficienti, un array di variabili, un array di valori e un array di segni. Il gioco contiene solo equazioni con coefficiente 1, ma la logica dovrebbe garantire il funzionamento (ammettendo che il sistema sia comunque lineare) di equazioni più complesse.
- SystemEquation, è una classe che rappresenta un sistema di equazioni tramite un array di Equation.

3.4 Interfaccia di comunicazione

Il SystemEquationFactory crea i sistemi di equazioni e li ritorna a chi li deve gestire graficamente. Gli oggetti grafici invocano metodi sulla GameLogic. Questa decide cosa far fare agli oggetti tramite segnali o metodi.

4 Lettura dei Dati (CSV)

Gestione Dati: Questa sezione illustra come il sistema gestisce la lettura e l'elaborazione dei file CSV.

4.1 Formato dei file supportati

Sono supportati i file .csv ma bisogna aggiungere .txt dopo .csv. Questo perché Godot fa confusione quando deve esportare dei csv. Le regole del file devono comunque rispettare le classiche dei csv. Per capire come sono organizzati, abbiamo scritto `how_to_read.csv.txt` dentro la cartella data.

4.2 Processo di lettura

Esiste una classe `DataManager` che si occupa della lettura dei dati dai csv. Non esiste nessun oggetto di questa classe ma solo un singleton autoload globale `SystemEquationFactory` che estende da `DataManager`. `DataManager` ha una funzione che ritorna il numero di round. Un'altra funzione legge il csv e ritorna i dati in un `Array`.

5 Panoramica Generale

Vista d'insieme: Questa sezione fornisce una panoramica completa del sistema e delle sue funzionalità.

5.1 Architettura del sistema

Il SystemEquationFactory crea i sistemi di equazione e li ritorna al main, il quale li aggiunge alla gui, in particolare al nodo Blackboard. Il GUI Layer è quindi in grado di gestire tutte le dinamiche grafiche del gioco e i suoi componenti sono strettamente legati alla GameLogic, la quale è chiamata nei punti cruciali, come all'inserimento di una risposta.

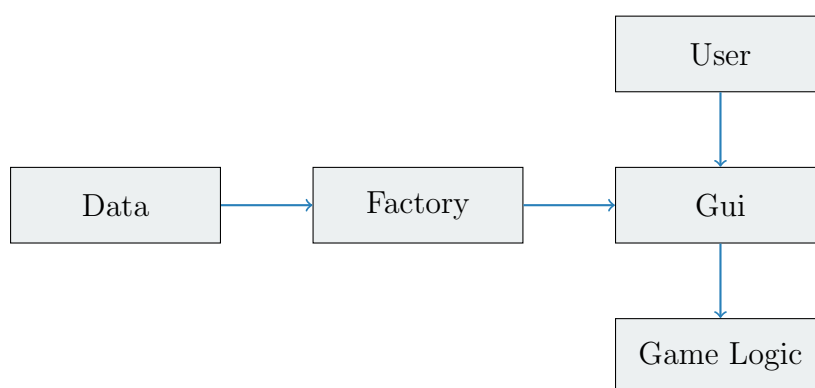


Figura 1: Architettura del sistema

5.2 Dipendenze esterne

Viene utilizzato il singleton JavaScriptBridge per utilizzare funzioni di JavaScript, necessarie per gestire alcune situazioni che avvengono durante l'uso sul Web. Ad esempio, la reindirizzazione verso l'URL dei giochi o la gestione dell'orientamento dello schermo da mobile.

6 Esportazione

Output Web: Questa sezione descrive la funzionalità di esportazione utilizzando template web di base.

6.1 Web build personalizzata

Abbiamo creato una build personalizzata per alleggerire l'engine originale di Godot, in modo da velocizzare il caricamento del gioco. Per esempio, abbiamo rimosso tutte le funzionalità e i nodi del 3D, che comunque non vengono mai utilizzati. La build è `godot_web_simple_template.zip`.

6.2 Funzionalità della shell

Abbiamo creato una shell personalizzata per migliorare la grafica del caricamento e implementare alcune funzionalità extra. La shell offre una schermata di caricamento trasparente. Al centro c'è il nome del gioco preceduto e seguito da un'immagine di un frutto. Il font del nome e le immagini sono caricate dalla cartella "art/". Infine c'è una barra di caricamento. Quando il gioco è pronto viene caricato all'interno di un canvas con lo sfondo trasparente. L'esportazione supporta i pixel trasparenti grazie alla riga che modifica il config dell'Engine di Godot. La shell permette di passare da fullscreen a windowed o viceversa col tasto F11.

6.3 Processo di esportazione

Per esportare il gioco per la prima volta in versione web bisogna seguire alcuni step:

1. Aprire il menù **Project** → **Export**.
2. Selezionare la piattaforma **Web** e aggiungere un nuovo preset.
3. Selezionare la Shell HTML personalizzata (`custom_shell.html`).
4. Aprire le opzioni avanzate e selezionare il modello personalizzato (`godot_web_simple_template.zip`) nella sezione "Release".
5. Nella pagina "risorse", selezionare come modalità d'esportazione: Esporta tutte le risorse nel progetto. Inoltre aggiungere *.txt nei file da esportare.
6. Cliccare su esporta e scegliere la cartella di destinazione. Conviene creare una cartella vuota apposita. Allo stesso livello della cartella vuota deve essere presente la cartella `art/` necessaria a caricare le risorse utilizzate nella custom shell.

Dopo la prima esportazione, tutte le opzioni vengono salvate in `export_presets.cfg` ed è sufficiente aprire il menù **Project** → **Export** e cliccare su Esporta.

6.4 Formati di output

L'esportazione produce i seguenti file:

- **index.html** → shell principale del gioco.
- **index.js** → codice JavaScript di avvio del motore.
- **index.wasm** → modulo WebAssembly del motore Godot.
- **index.pck** → pacchetto delle risorse del progetto.
- **.png** → icone.
- **.js** → necessari per gli audio.

7 Rilascio

7.1 Git repo

github <https://github.com/Luca-Milioli/i-frutti-dell-ingegno.git> (selezionare il branch corretto)

bitbucket <https://317076@bitbucket.org/melazetasrl/spreafico-i-frutti-dellingegno.git>